

Operating System (303105251)

Prof. Ami Tank Assistant Professor,
Computer Science & Engineering





CHAPTER-5.1

Memory Management



Topics

1. Memory Management: Basic Concept.
2. Logical and Physical address map.
3. Memory Allocation: Contiguous Memory Allocation.
4. Fixed and Variable Partition.
5. Internal and External Fragmentation and Compaction.
6. Paging: Principle of Operation.
7. Page Allocation.
8. Hardware support for paging



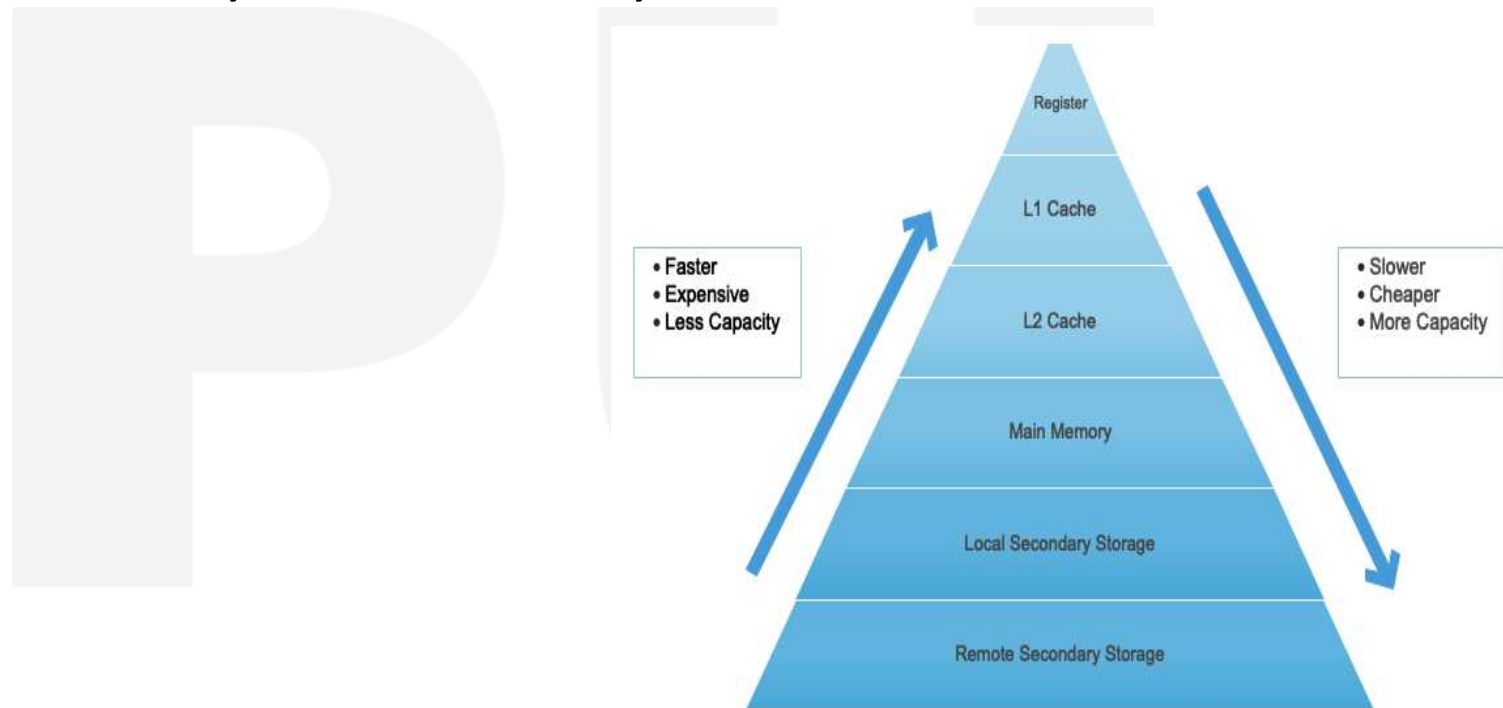
What is Memory Management?

- Memory Management is an important characteristic of the Operating System. This comes beneath one of the two main capabilities of an Operating System, useful resource management. **Main memory (RAM)** is where maximum of the programs run. It is one of the most important matters people search for whilst buying a new cellphone or a new laptop.



What is Memory hierarchy ?

- In computer system architecture memory Hierarchy is part to arrange the memory in a such a way it can minimize access time.





Logical and physical address

- **Logical address** is CPU generated address and it generate while program is running.
- It considers as a virtual address and it not exist physically.
- It use as a reference for access Physical memory point by CPU.
- In a memory physical location identifies by **Physical address**.



Logical and physical address

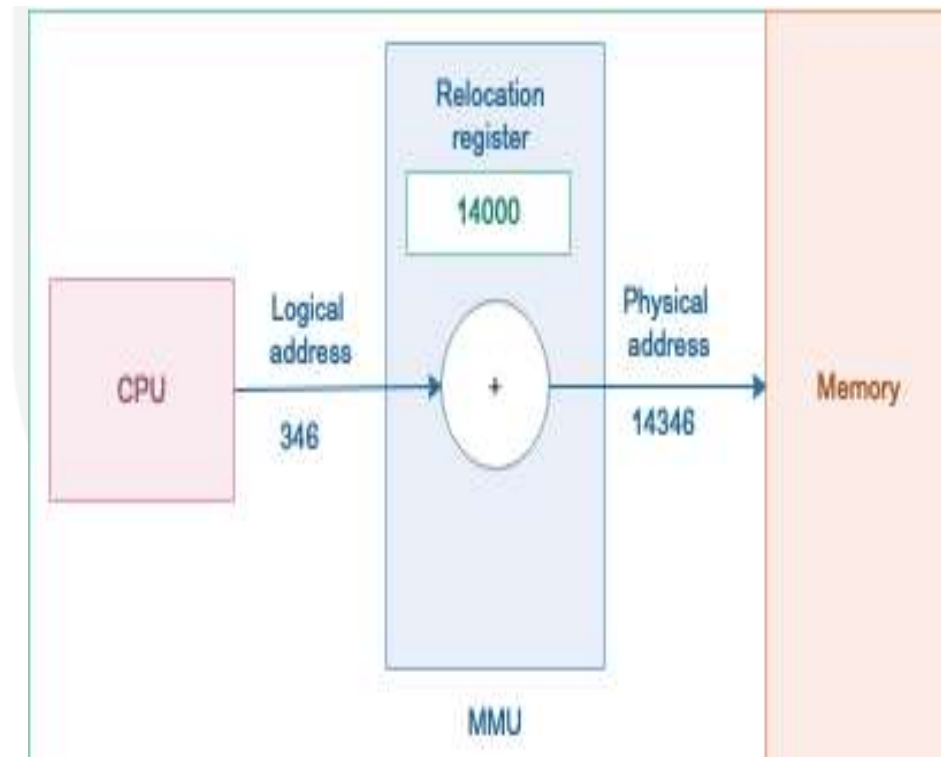
- Physical address not directly access by the user but it can be accessible by its corresponding logical address.
- The term Physical Address Space is utilized for every physical location relating to the logical addresses in a Logical location space.
- MMU computes **Physical address**.



What is Memory hierarchy ?

- It is a hardware device which uses to map virtual address to physical address.

P



Multistep processing of user program

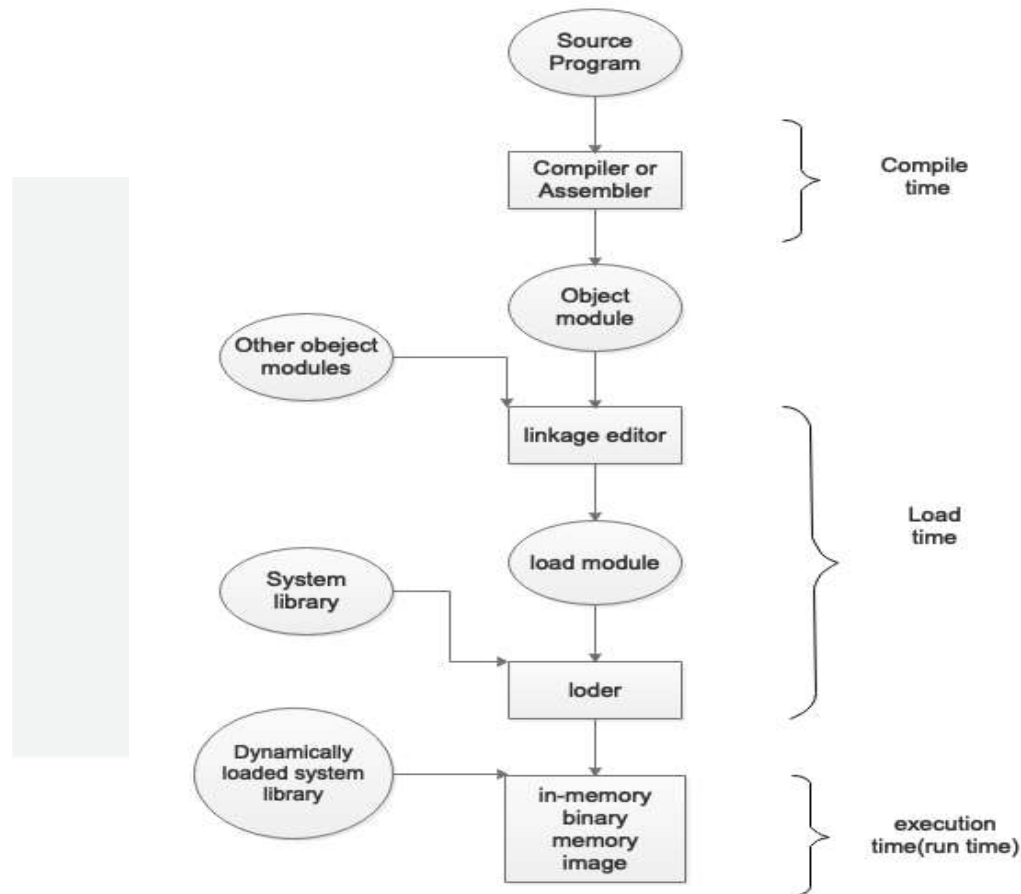


Figure – 3 Diagram of



Base and Limit register: Memory Protection

- Set of address that a process can use to address memory called an address space. It is range of valid addresses in a memory which are available for a program or process.
- Start address of program **is base register**.
- **Length of program is limit register.**
- **Base and limit register only modify by OS**

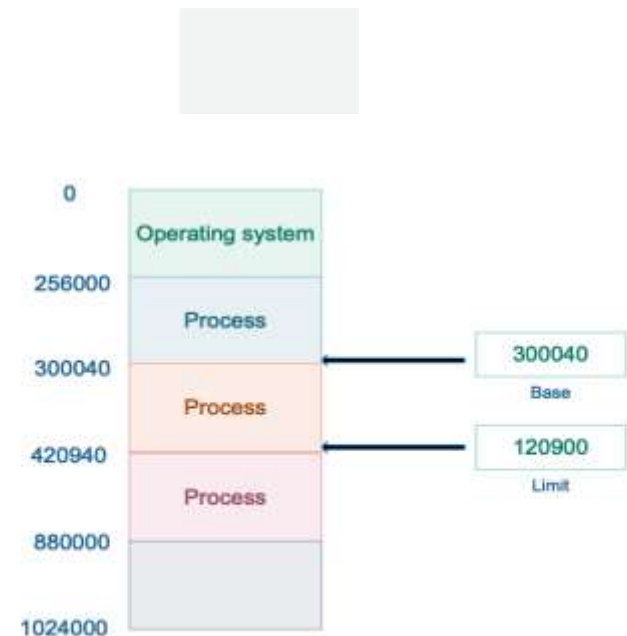


Figure – 4 a based

Static vs Dynamic Loading

Static	Dynamic
1. Entire program load into main memory before program start execution.	1. On demand it load program into memory
2. Memory utilization is inefficient because it require or not it load whole program into main memory.	2. Memory utilization is efficient because it load program on demand.
3. Fast execution of program	3. Slower execution of program
4. Every time it takes load time when program load into main memory	4. Perform at a run time by operating system
5. Static linking is applied if the static loading used accordingly	5. dynamic linking is applied if the dynamic loading used accordingly
6. To start execution absolute data and program load into memory	6. Bit by bit loading of data and information in run time



Swapping

- For temporarily any process can be swapped out of memory to backing store and take back into memory for further execution.
- **Backing store** : disk large enough so that can accommodate copies of all memory images for all users.
- **Roll out roll in**: for priority based scheduling algorithm swapping variant used; to load high-priority process and to get execute lower priority process need to swapped out.
- Ready queue maintain by the system for ready -to-run processes which have memory images on disk.



Schematic View of Swapping

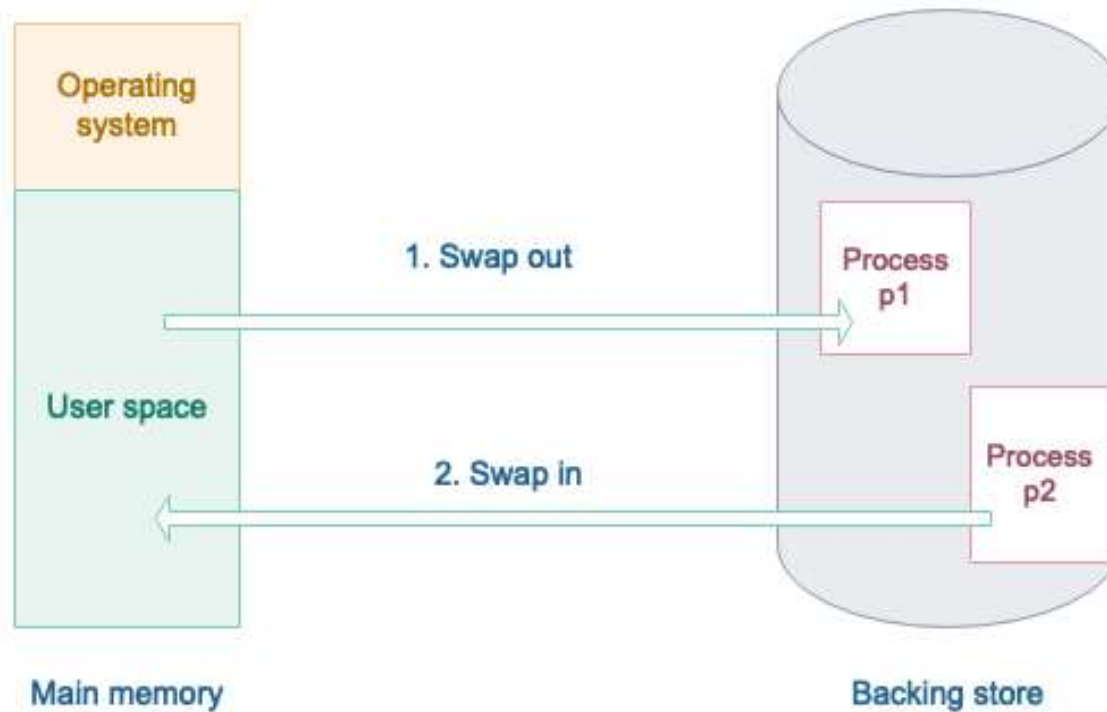
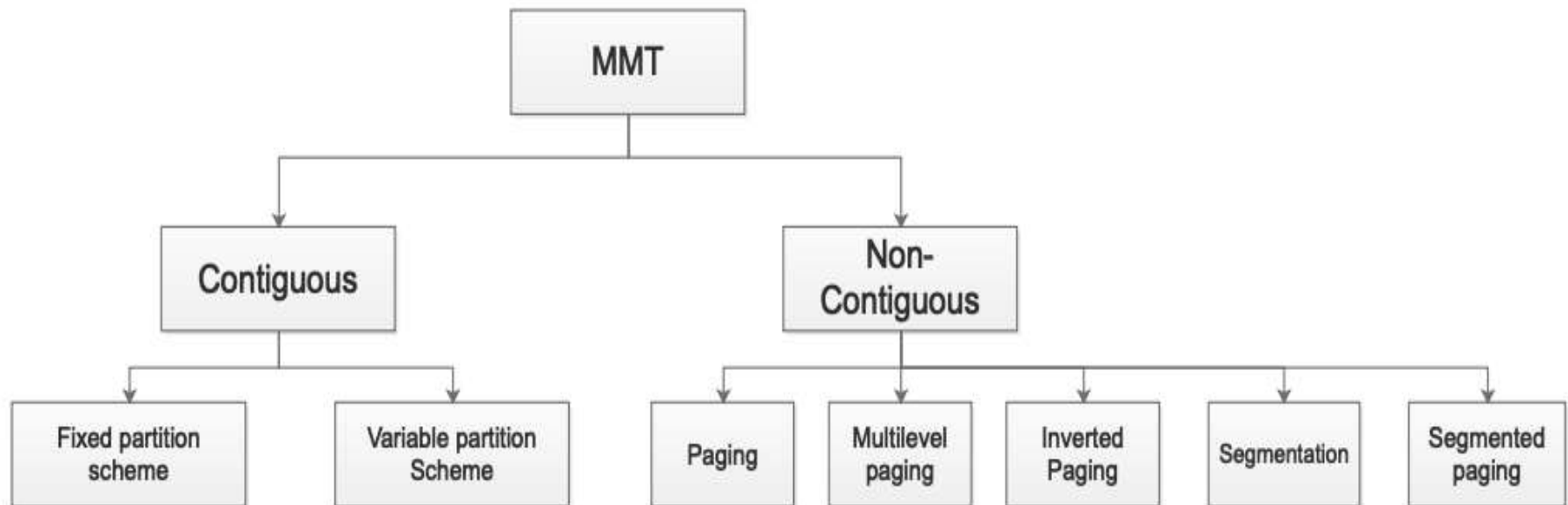


Figure – 6 Schematic
view Swapping [6]



Memory Management Techniques (MMT)



Memory Allocation: Contiguous Memory Allocation

- Memory allocation is a mechanism by which computer programs and services are assigned with physical or virtual memory space.
- Memory allocation is the process of reserving a partially or entirely portion of computer memory.
- There are 2 types of memory allocation:
 - 1. Contiguous memory allocation**
 - 2. Non – Contiguous memory allocation**



Contiguous Memory Allocation

- Contiguous memory allocation is known as a memory allocation model in which a single contiguous adjacent area is allocated in the memory for each program.
- When a process needs to execute, memory is requested by the process.
- The size of the process is compared with the amount of contiguous main memory available to execute the process.



Contiguous Memory Allocation

- If enough contiguous memory is found, the process is allocated memory to start its execution.
- Otherwise, waiting processes are added to a queue until there is sufficient free contiguous memory available.



Contiguous Memory Allocation

- In operating systems, the contiguous memory allocation scheme can be implemented with the aid of two registers, known as base and limit registers.

- Main memory usually has two partitions :

Low Memory : Operating system resides in this memory.

High Memory : User processes are held in high memory.



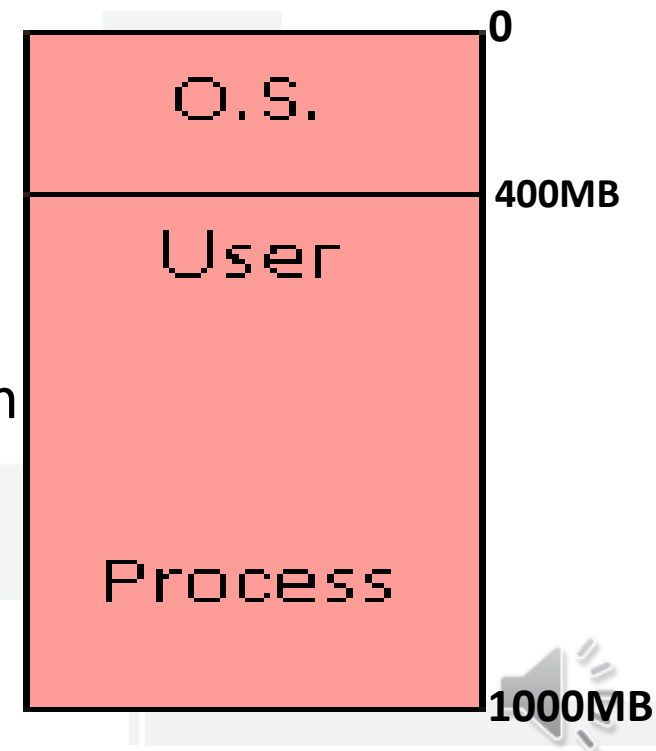
Contiguous Memory Allocation

- Partitioning of memory can be done in the following ways:
 1. Single Process Monitor
 2. Multiprogramming with Fixed Partition
 3. Multiprogramming with Variable Partition



Single Process Monitor

- Main memory is dividing into the two parts. One is part of O.S. and second for user process area.
- For example: Here main memory 0-400MB space is reserve for the Operating System.
- Second part is part for user process in which user's process will be loaded into memory.



Single Process Monitor

- Only one process can load to memory in Single Process Monitor. The next process will be loaded into memory when one process is complete.
- Single process monitor does not support multiprocessing but only supports uniprogramming.



Multiprogramming with Fixed Partition (MFT)

- The main memory is divided into multiple partitions to support the multiprogramming.
- The number of partitions are fixed but the size of each partition can be different.
- Each process will store into these partitions contiguously.





Multiprogramming with Fixed Partition (MFT)

- Here only one process is allowed into one partition.
- The degree of multi-programming is equivalent to the number of partitions as there can be many processes in the main memory at one time

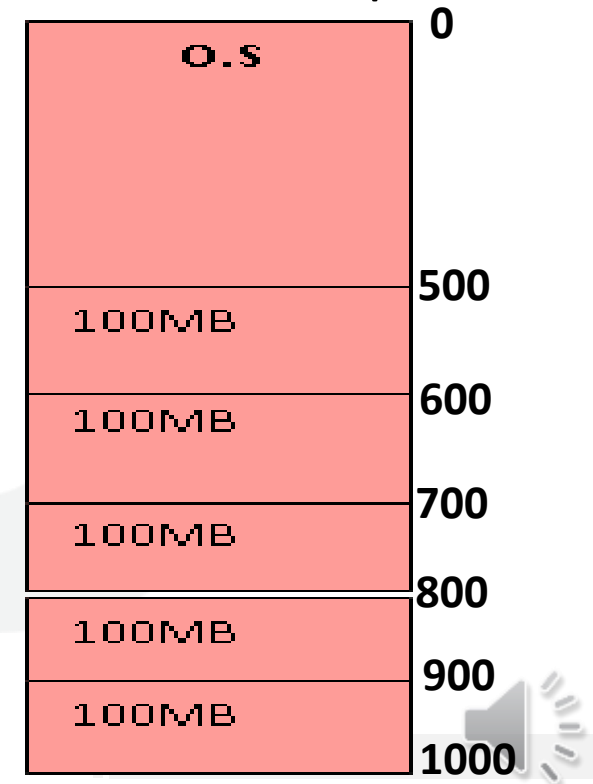


Multiprogramming with Fixed Partition (MFT)

- In fig.2 memory is divided into 5 equal size partitions. Each partition of 100MB.

Advantages of Fixed Partition:

1. Easy to implement.
2. Little OS overhead.



Multiprogramming with Fixed Partition (MFT)

Disadvantages of Fixed Partition:

1. Internal Fragmentation
2. External Fragmentation
3. Limit process size
4. Limitation on Degree of Multiprogramming



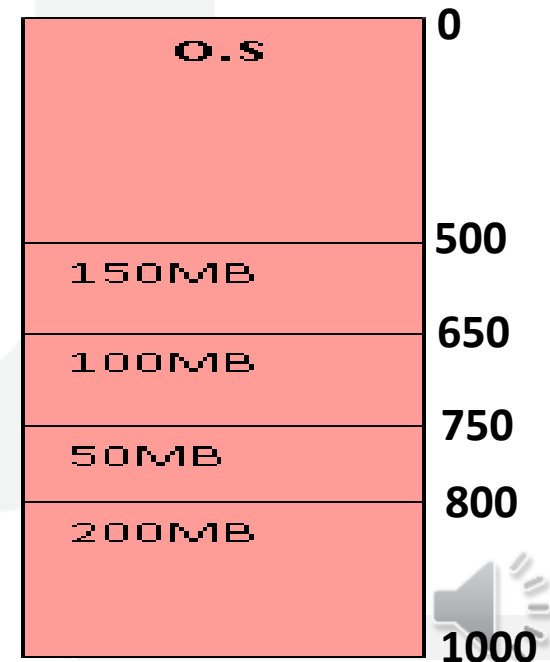
Problems with Fixed Partitioning

- Main problem with this arrangement is fragmentation.
- Fragmentation is the presence of unusable areas in the computer memory
- There are two types of Fragmentation
 1. Internal Fragmentation
 2. External fragmentation



Multiprogramming with Variable Partition(MVT)

- Partitions are created dynamically to avoid the difficulties of the fixed partitions.
- No partition is fixed in memory in dynamic partitions, but the partition is only created when the process is loaded into memory.



Multiprogramming with Variable Partition(MVT)

- In figure above memory is divide into 4 unequal size partitions.
Partition size is not equal.
- Here the not problem of internal fragmentation. But here the problem of external fragmentation.



Multiprogramming with Variable Partition(MVT)

Advantages of Variable Partition:

- 1.No internal fragmentation.
- 2.Efficient use of memory.

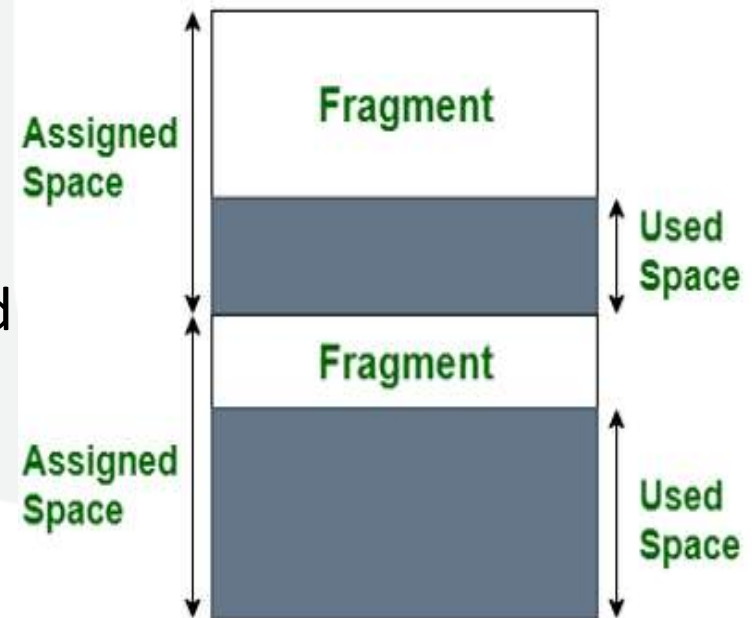
Disadvantages of Variable Partition:

- 1.Problem of external fragmentation.
- 2.Not easy to implement.



Internal Fragmentation

- Internal fragmentation occurs when the memory is divided into fixed sized blocks.
- Whenever a process request for the memory, the fixed sized block is allocated to the process.



Internal Fragmentation

- In case the memory assigned to the process is somewhat larger than the memory requested, then the difference between assigned and requested memory is the **Internal Fragmentation**.
- Internal fragmentation occurs when fixed sized memory blocks are allocated to the process without concerning about the size of the process.



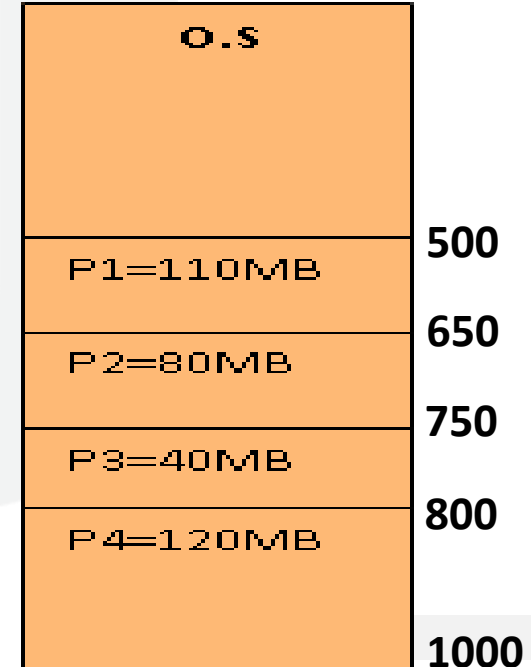
Internal Fragmentation

- The diagram clearly shows the internal fragmentation because the difference between memory allocated and required space or memory is called Internal fragmentation.



Example of Internal Fragmentation

- Here p1, p2, p3, p4 is loaded into memory.
- Process P1 is loaded into first partition whose size is 150MB and size of process P1 is 110MB, so in partition 40MB space is waste.
- This is called **Internal Fragmentation**.



External Fragmentation

- When the process is loaded or removed from the memory, a free space is created. This free space creates an empty space in the memory which is called external fragmentation.
- When the memory assigned to the process dynamically based on process request.

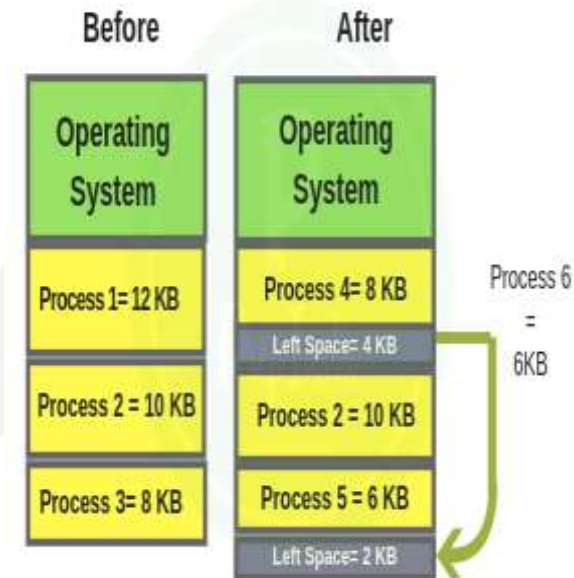


Figure - 13 Example of

External Fragmentation

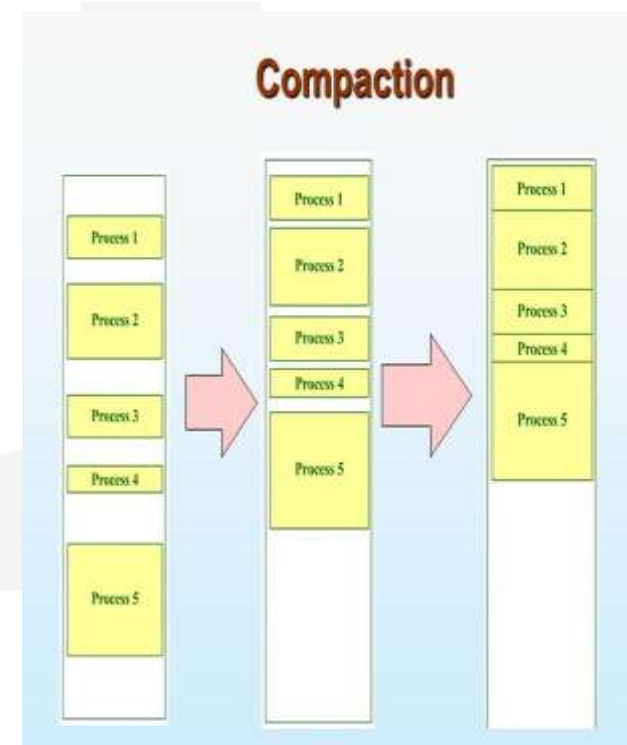
- Memory is partitioned into variable size blocks.
- External fragmentation can be eliminated by compaction, paging, and segmentation so that memory can be allocated in a non-contiguous manner to a process.





Compaction

- Compaction is **one of the technique to avoid External Fragmentation.**
- Move all the processes towards the top or towards the bottom to make free available space in a single contiguous block is known as **compaction.**



Compaction

- Compaction is undesirable to implement because it interrupts all the running processes in the main Memory.
- The other technique to avoid external fragmentation is to implement non-contiguous memory allocation techniques.



Paging: Principle of Operation

- In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages.
- The main idea behind the paging is to divide each process in the form of pages.
- The main memory will also be divided in the form of frames.



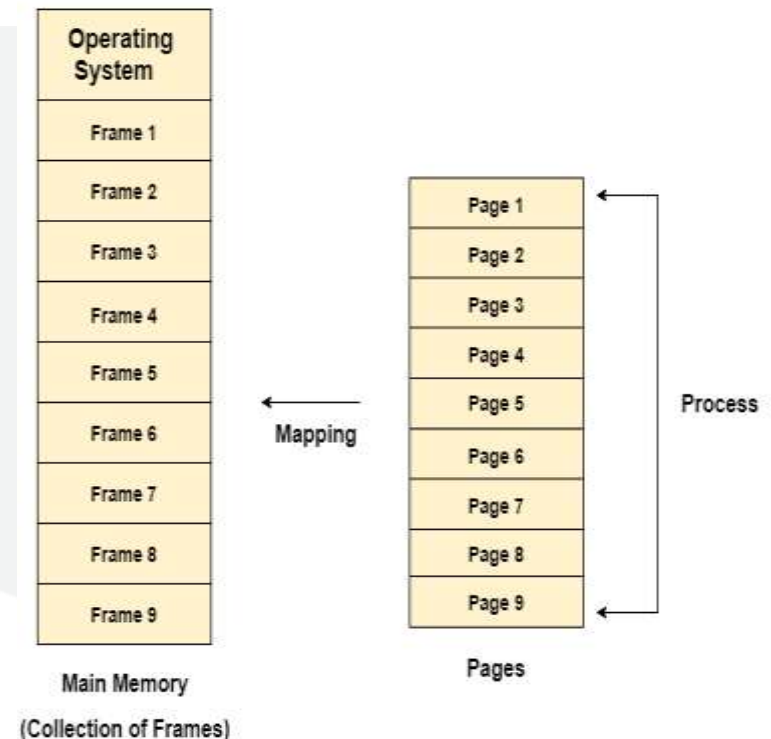
Paging: Principle of Operation

- One page of the process is to be stored in one of the frames of the memory.
- The pages can be stored at the different locations of the memory, but the priority is always to find the contiguous frames or holes. Different operating system defines different frame sizes.



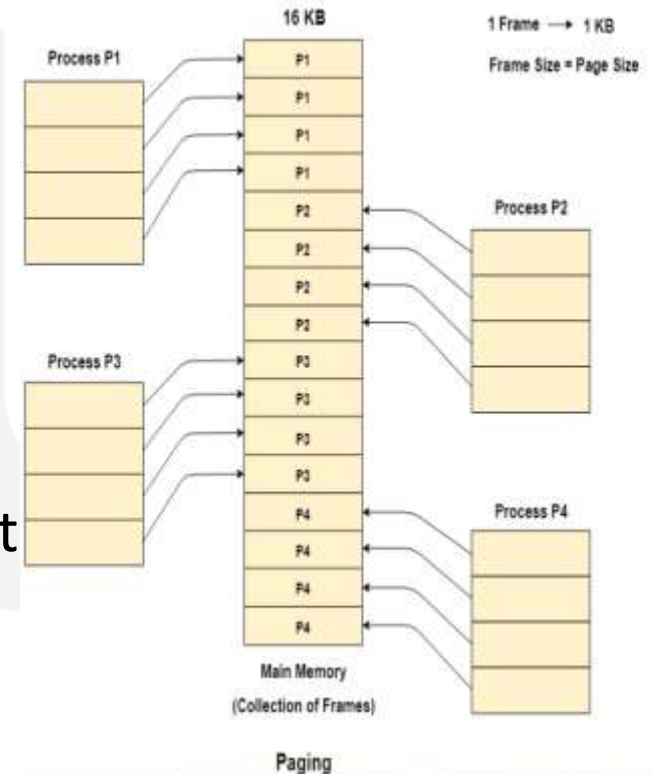
Paging: Principle of Operation

- The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.



Example of Paging

- Let us consider the main memory size 16 Kb and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each.
- There are 4 processes in the system that are P1, P2, P3 and P4 of 4 KB each.



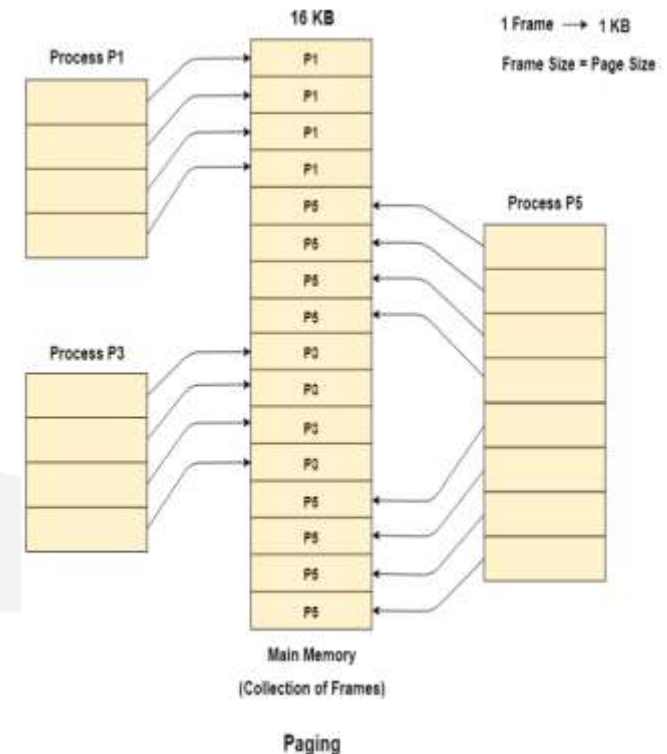
Example of Paging

- Each process is divided into pages of 1 KB each so that one page can be stored in one frame.
- Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way.



Example of Paging

- Let us consider that, P2 and P4 are moved to waiting state after some time.
- Now, 8 frames become empty and therefore other pages can be loaded in that empty place.



Example of Paging

- The process P5 of size 8 KB (8 pages) is waiting inside the ready queue.
- Given the fact that, we have 8 non contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.



Memory Management Unit

- The purpose of Memory Management Unit (MMU) is to convert the logical address into the physical address.
- The logical address is the address generated by the CPU for every page while the physical address is the actual address of the frame where each page will be stored.



Memory Management Unit

- The logical address has two parts:
 1. Page Number
 2. Offset
- Memory management unit of OS needs to convert the page number to the frame number.





Physical Address Space

- Physical address space in a system can be defined as the size of the main memory.
- It is really important to compare the process size with the physical address space. The process size must be less than the physical address space.
- **Syntax of Physical Address Space is:**

Physical Address Space = Size of the Main Memory





Example of Physical Address Space

If, physical address space = 64 KB = 2^6 KB = $2^6 \times 2^{10}$ Bytes = 2^{16} bytes

Let us consider,

word size = 8 Bytes = 2^3 Bytes

Hence,

Physical address space (in words) = $(2^{16}) / (2^3) = 2^{13}$ Words

Therefore,

Physical Address = 13 bits



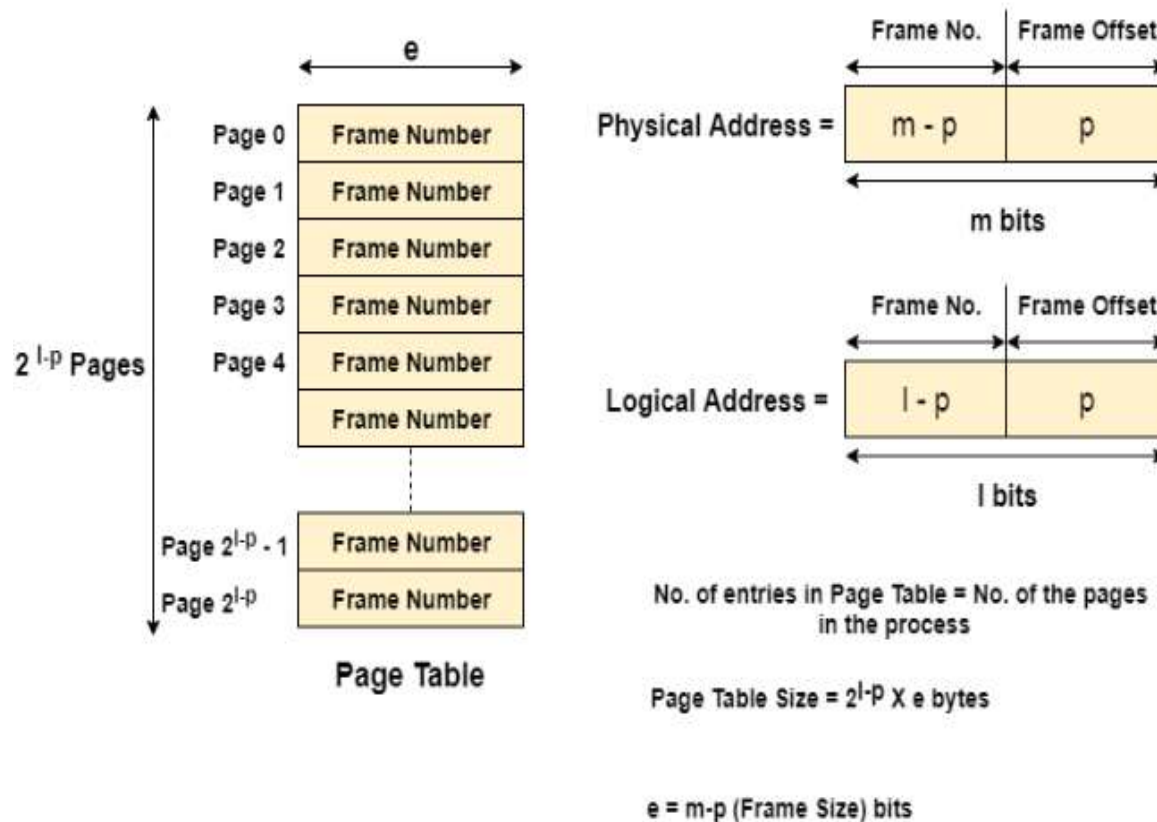
Page Table

- Page Table is a data structure used by the virtual memory system to store the mapping between logical addresses and physical addresses.
- Logical addresses are generated by the CPU for the pages of the processes therefore they are generally used by the processes.
- Physical addresses are the actual frame address of the memory.
- They are generally used by the hardware or more specifically by RAM subsystems.

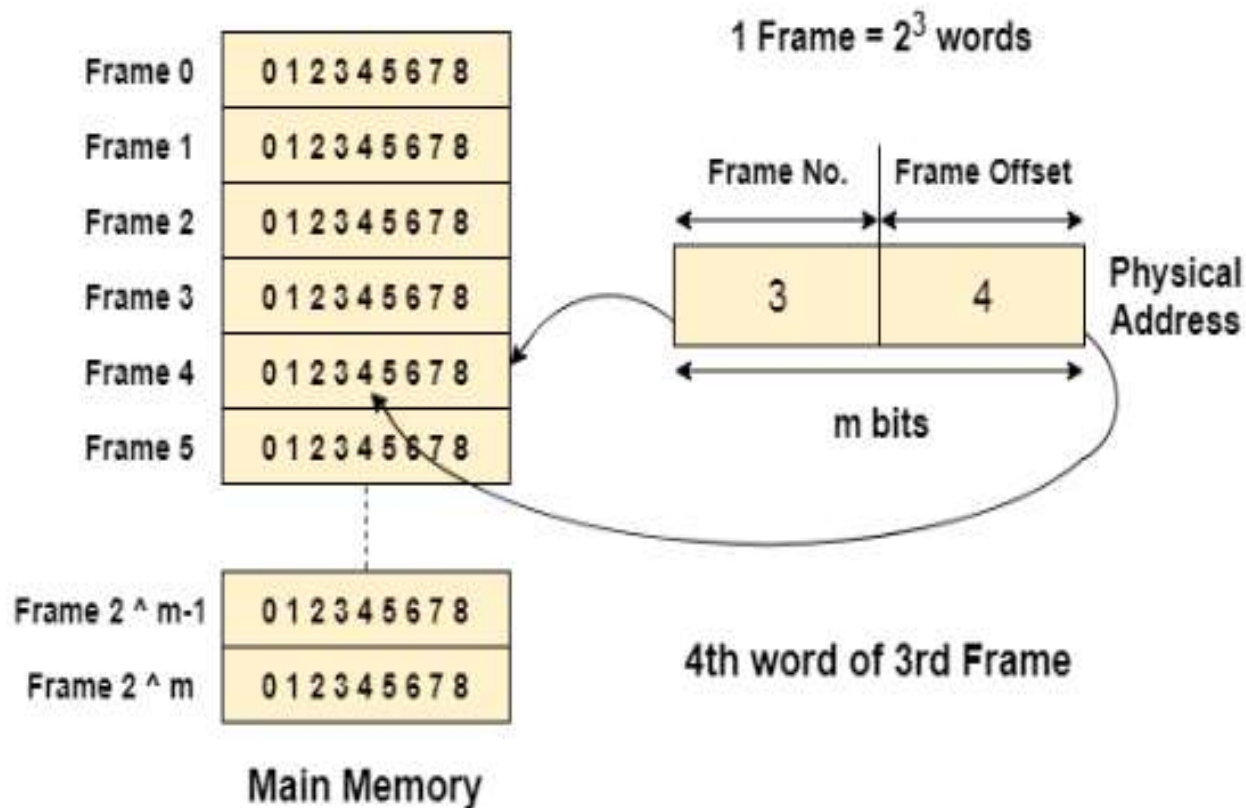




Representation of Physical and Logical Address



Example of Page Table



Page Allocation

- Paged allocation in memory management divides computer physical memory (Random Access Memory – RAM) into fixed size of memory units called page frames (memory block size).
- Also divides virtual address space to size of the pages.



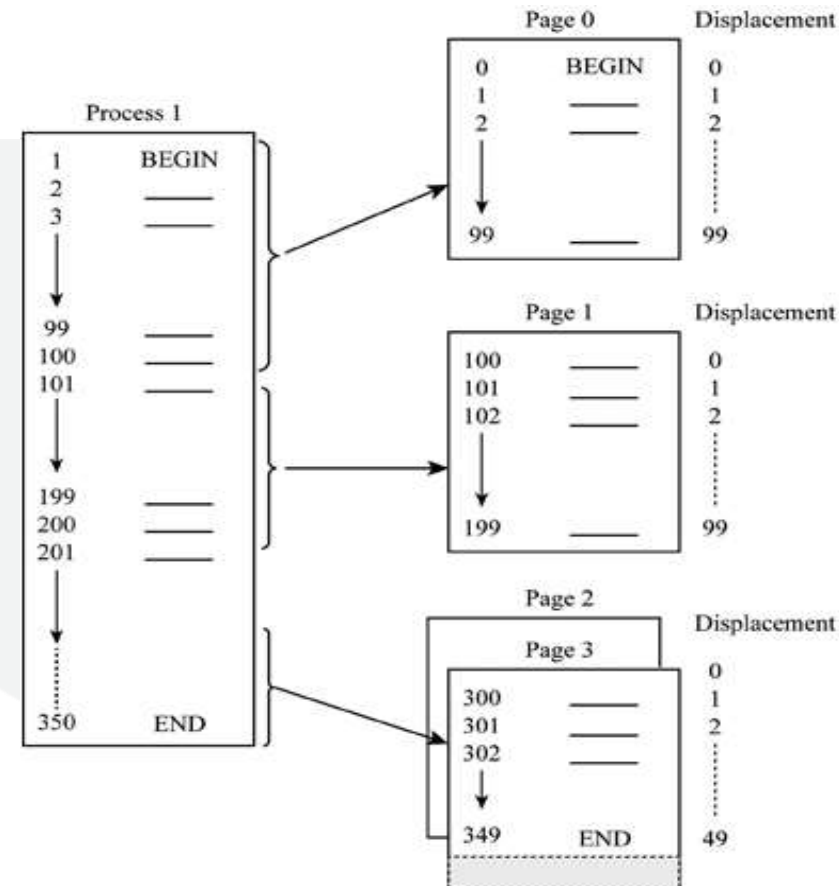
Page Allocation

- The main advantage of using paged allocation is, an empty page frame can be used by any job however, need to keep track of number of pages and where the pages of individual job are located in memory.
- In other words memory can be used efficiently and each job runs on its own address space. Frames, pages and the mapping between the two is shown in the .



Example of Page Allocation

- From figure, the size of each page is 100 memory locations.
- The memory utilized by the process1 in the given figure is 350 memory locations where it takes 4 pages to store 350 memory locations.



Example of Page Allocation

- That is, in memory block 0 or page 0 holds first 100 memory locations of data.
- In second page, memory locations of 100 to 199 are stored and so on.



Hardware Support

- Each OS has its own method for storing page tables.
- Page table for each process is there.
- Pointer to the page table is stored with other register values in PCB.
- The h/w implementation of page table can be done in several ways.

Hardware Support

- 1) The page table is implemented as a set of dedicated registers.
 - The main use of registers for the page table is satisfactory if the page table is reasonably small.
 - But current systems have Page table size that is very large, so use of only registers is not feasible.

Hardware Support

2) Using Translation look-aside buffer (TLB).

- The problem of previous approach is the time required to access a memory, with this scheme 2 memory access required like, a P.T. is kept in M.M., and a page- table based register (PTBR) points to the page table.
- The solution of this problem is to use a special, small, fast-lookup hardware, known as translation look-aside buffer (TLB).

TLB (Translation Look-Aside Buffer)

- It is associative, high speed memory.
- Each entry in TLB consists of 2 parts:
 - 1) a key (tag)
 - 2) a value.

TLB (Translation Look-Aside Buffer)

- TLB used with P.T. in following way:
 - It contains a few of the page-table entries.
 - When a logical address is generated by the CPU, its page no. is presented to the TLB.
 - **If the page no. is found in TLB**, its frame no. is immediately available and is used to access memory.
 - **If the page no. is not in TLB** (known as TLB miss), a memory reference to the page table must be made.
 - When the frame no. is found then we can use it to access memory.

Paging Hardware with TLB

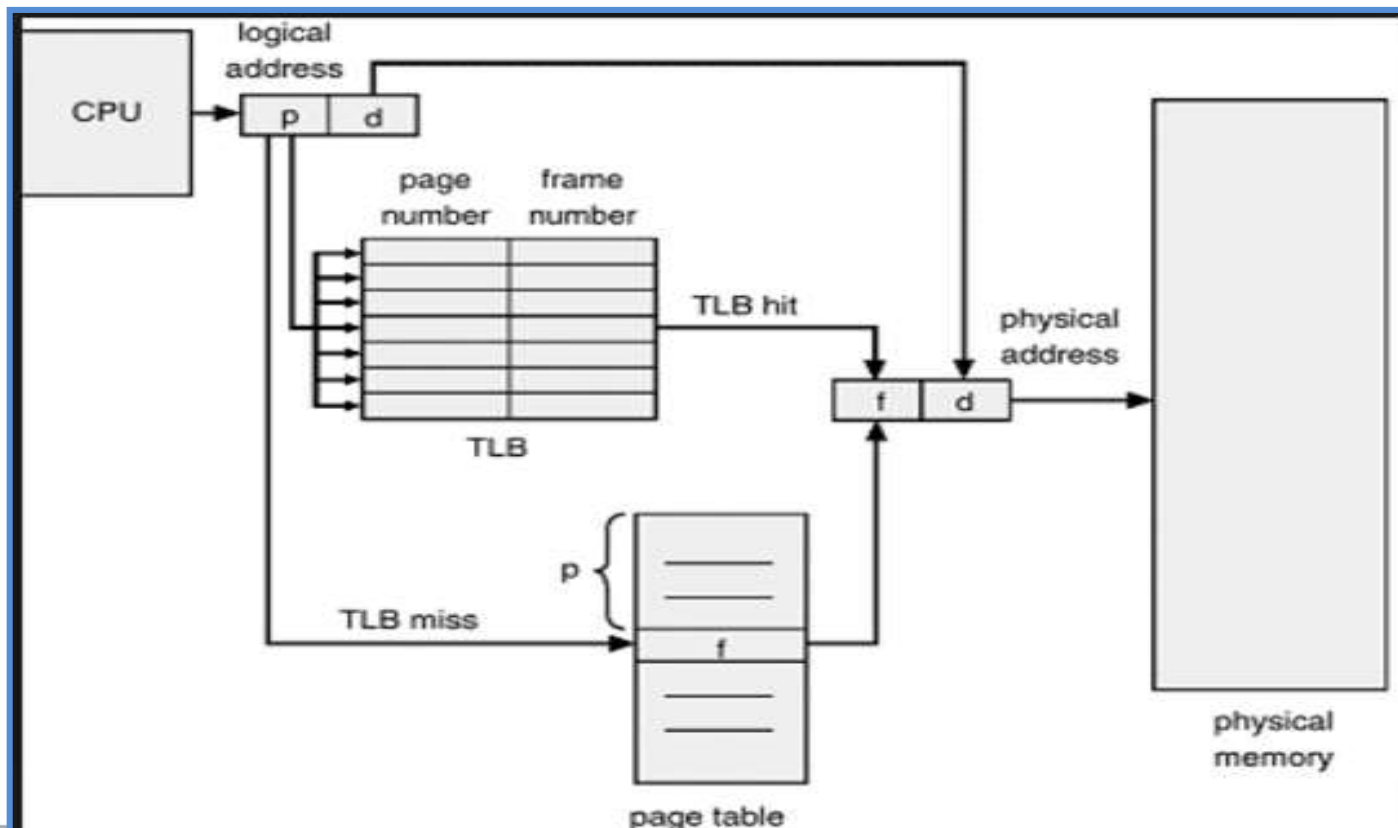


Figure – 22 Paging

Paging Hardware with TLB

- Now add that page no. and frame no. into TLB so that next time it would be helpful to access a physical memory.
- If the TLB is full of entries, some of them can be removed. This decision has been taken by using LRU algorithm.
- Some TLBs allowed entries to be wired down that means it can't be removed.
- Some TLBs store address-space identifiers (ASIDs) in each TLB entry.

Paging Hardware with TLB

- ASID –uniquely identify each process and used to provide address space protection to corresponding process.
- It ensures that the ASID for the currently running process must matches with the virtual page that it is now resolving.
- If it doesn't match , it is known as TLB miss.
- An ASIDs allow TLB to contains different process simultaneously.
- If the TLB does not support separate ASIDs, then every time a different page table is selected, and the previous TLB must be erased.

Paging Hardware with TLB

- The percentage of times that a page is found in TLB is known as hit ratio.

For Example:-

- 80 % hit ratio means we find desired page in 80 percent of time.
- It takes 20 nanosecond to search the TLB and 100 nanoseconds to access memory. So mapping requires total 120 nanoseconds total.
- If its not in TLB then, 20 ns for TLB search, 100 ns for memory access for page table, then 100 ns for mapping, so total 220.
- Effective Access time = $(0.80 * 120) + (0.20 * 220)$
 $= 140 \text{ ns}$

Paging Hardware with TLB

Example:- 2

- If hit ratio increase as 98 %
- Effective Access time = $(0.98 * 120) + (0.02 * 220)$
= 122 ns

Advantages of Paging

- Memory allocation and de-allocation is very fast.
- It is noncontiguous memory allocation so we do not require contiguous memory space for one process.
- No external fragmentation problem.
- Swap-in and Swap-out operations are performed fast.

Disadvantages of Paging

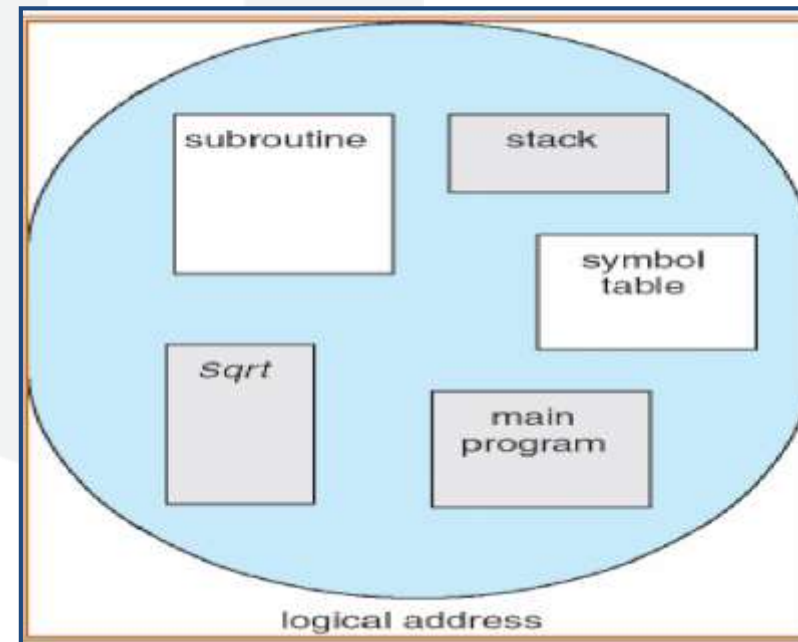
- This technique is suffering with internal fragmentation.
- **For Example:-**

If pages are 120 bytes, a process of 400 bytes requires 3 pages + for 40 bytes it requires again 1 page. So total 4 pages will be allocated.

$(120 - 40) = 80$ bytes size will be free and its internal fragmentation.
- It requires more size of memory to store page table. As the page number increases the page table size is increase.

Segmentation

- Segmentation is a memory-management scheme that supports user view of memory and contains segment.
- In fig. we can see that memory is divided into multiple segments like, subroutine, stack, code, heap, etc.

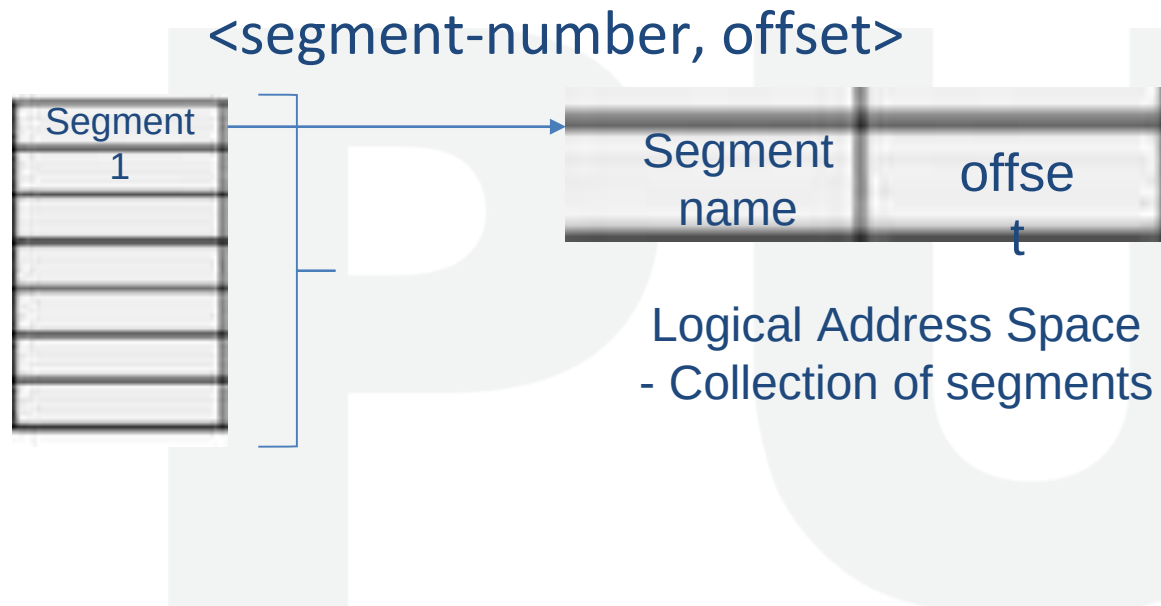


Segmentation

- Users or programmers specify each address by 2 quantities:
 - 1) a segment name
 - 2) an offset. (contrast to the paging)
- For easiest way of implementation, segments are numbered and referred by segment no. rather than name.

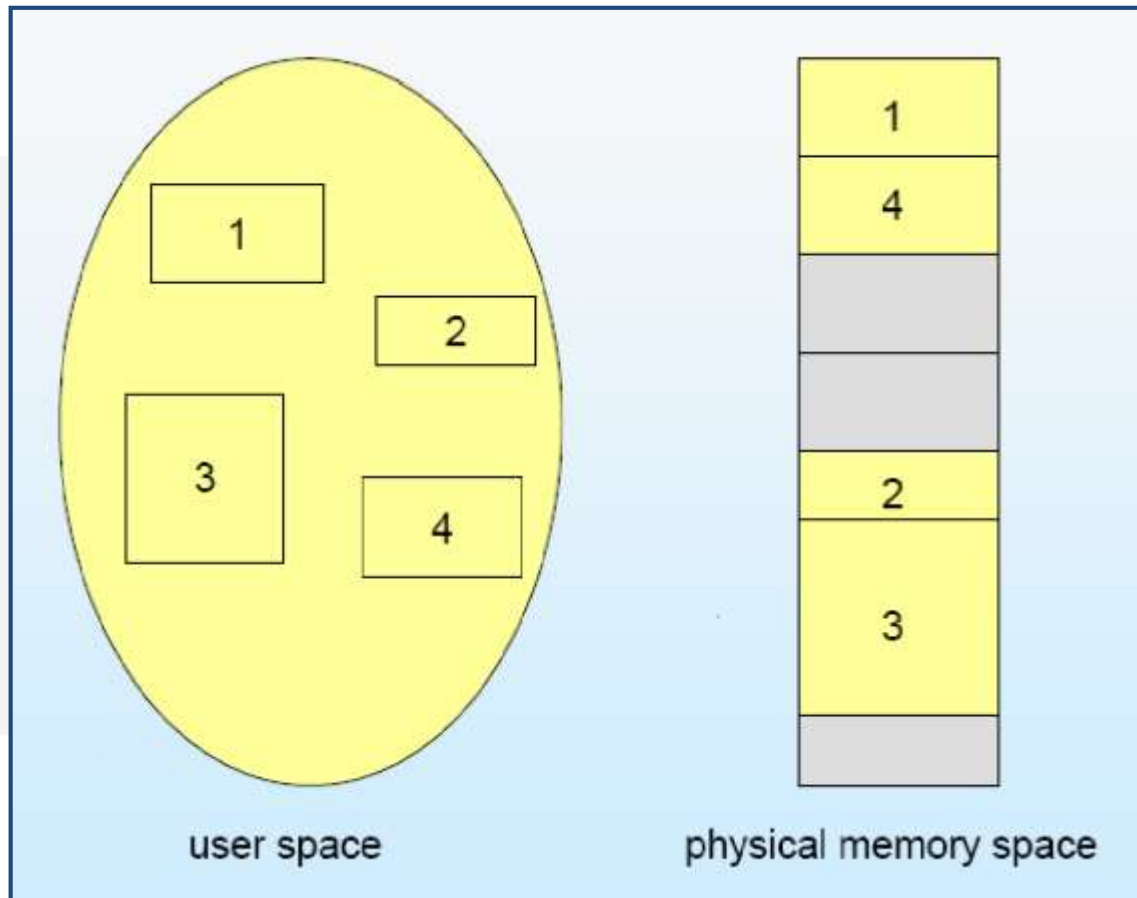
Segmentation

- So logical address consists of 2 tuple:

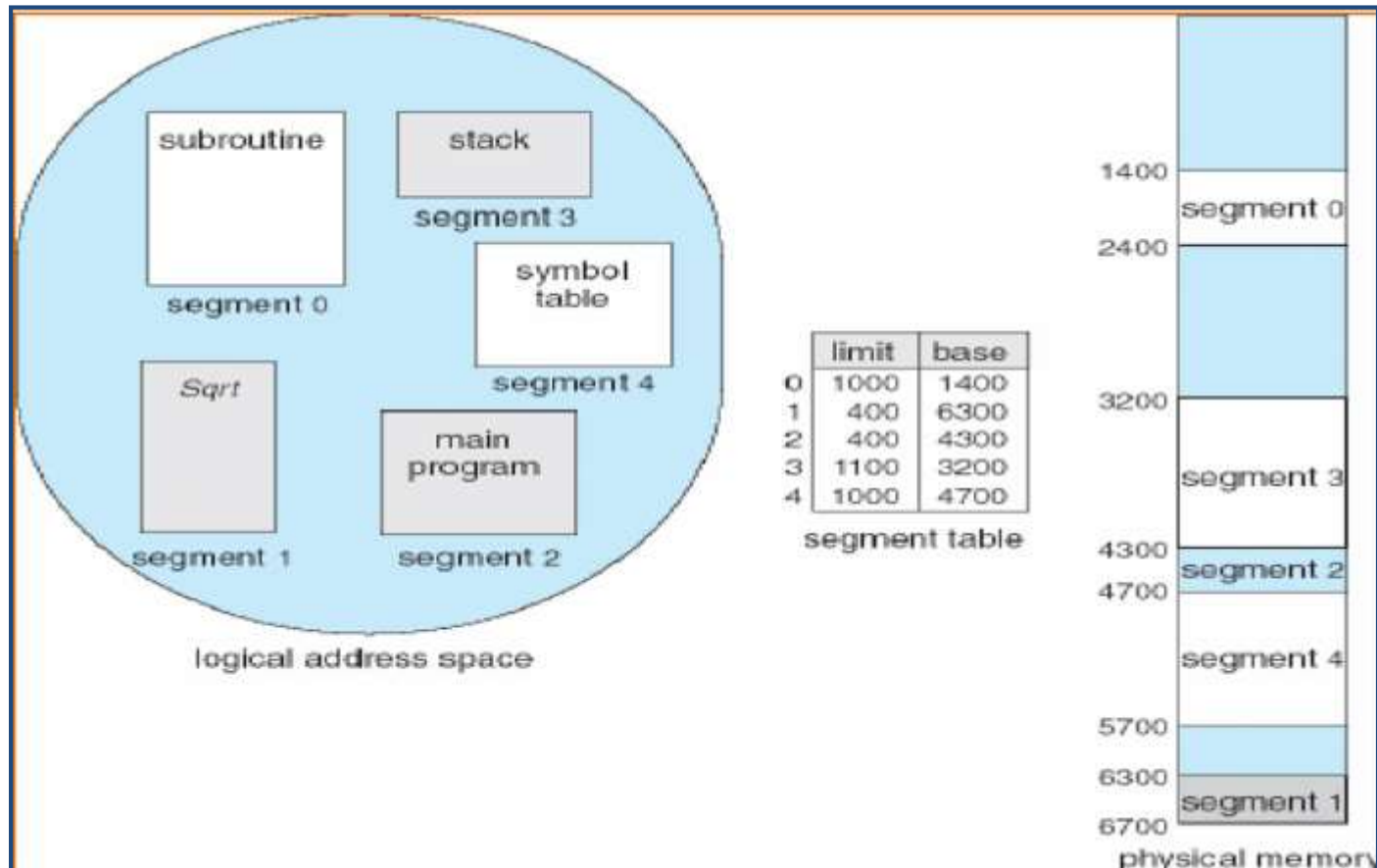


- Here, Segment-number is used to identify a segment.offset is an original location within a segment.

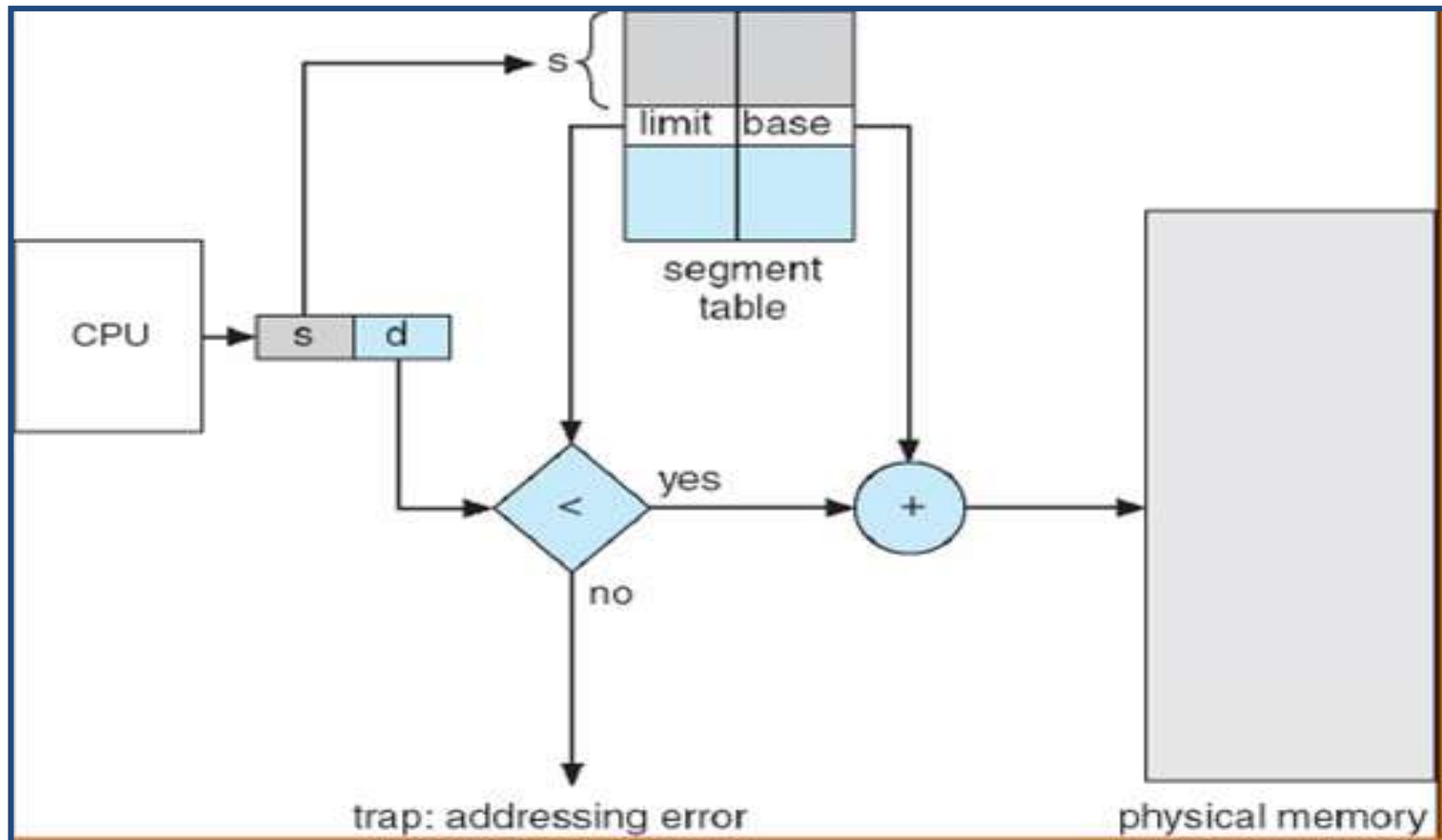
Segmentation



Example of Segmentation



Hardware Implementation of Segmentation



Hardware Implementation of Segmentation

- A logical address consists of 2 parts: s & d.
- Here, S is used as an index into segment table
 - a value of D must be lies between 0 and segment limit.
 - If its not, it will be trapped to OS.

For Example,

- Segment 2 is 400 bytes long. Begins at 4300.
- It will check $53 < 0 - \text{limit (i.e. 400)}$
- So that, a reference to byte 53 of segment 2 is mapped as
$$4300 + 53 = 4353.$$
- now check for 1222 byte for segment 0 .
- $1222 < 0 - 1000$, which is false so it will be trapped to OS.



Difference between Segmentation and Paging

	Segmentation	Paging
1	Program is divided into variable size segments.	Program is divided into fix size of pages.
2	Segmentation is slower than paging.	Paging is faster than segmentation.
3	Segmentation is visible to the user.	Paging is invisible to the user.
4	Segmentation eliminates internal fragmentation.	Paging suffers from internal fragmentation.
5	Segmentation suffers from external fragmentation.	There is no external fragmentation.
6	OS maintain a list of free holes in main memory.	OS must maintain a free frame list



Advantages of Segmentation

- Segmentation is useful for memory management.
- Using segmentation users can partition their programs into number of subtask i.e. modules which are operate independently from each another.
- Segments increases data sharing between two processes.
- Segmentation allows to extend the address ability of a processor
- Segmentation provides facility to separate the memory areas for heap, code, stack, data, etc.

Disadvantages of Segmentation

- In segmentation external fragmentation is mostly present.
- Costly algorithm.
- Segmentation finds free memory area big enough.
- When we use paging it generates list of free pages.
- Implementation is very complex.



References

- [1] https://www.researchgate.net/publication/319529366_Emerging_NVM_A_Survey_on_Architectural_Integration_and_Research_Challenges/figures?lo=1
- [2] <https://www.techtud.com/short-notes/logical-vs-physical-address-space>
- [3] <https://basicittopic.com/memory-management/>
- [4] <https://professormerwyn.wordpress.com/tag/logical-address-space/>
- [5] https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/8_MainMemory.html
- [6] <https://www.inf.ed.ac.uk/teaching/courses/os/slides/09-memory16.pdf>





References

- [7] <https://www.geeksforgeeks.org/implementing-non-contiguous-memory-management-techniques/>
- [8] Operating Systems: Internals and Design Principles, by William Stallings.
- [9] Operating Systems: Design and Implementation, Textbook by Andrew S. Tanenbaum.
- [10] Operating System Concepts by Galvin
- [11] Modern Operating Systems, Andrew S. Tanenbaum
- [12] Operating system concepts by Abraham Silberschatz. Peter B Galvin. Gerg
- [13] OPERATING SYSTEMS MEMORY MANAGEMENT by Jerry Breecher





References

[14]Operating System Generations. Tutorialspoint.

<https://www.tutorialspoint.com/operating-system-generations>

[15]Image source www.google.com



× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

